

GuardianIQ Engineering Handbook

Volume 2A: UX Foundation & Information Architecture

Developer-ready UI/UX technical implementation specification

Platform	GuardianIQ
Document	Engineering Handbook - Volume 2A
Focus	UX Foundation, Information Architecture, Navigation and Layout Standards
Audience	Frontend Developers, Backend Developers, QA, PM, Product Owners
Version	v1.0
Prepared For	GuardianIQ Product & Engineering Team

Core design statement: GuardianIQ UX must make governance visible, relationships understandable, decisions explainable and audit traceable without overwhelming enterprise users.

Table of Contents

1. Purpose and Scope
2. GuardianIQ UX Philosophy
3. Product Design Principles
4. User Personas and Role-Based UX
5. Information Architecture
6. Navigation Architecture
7. Module Hierarchy and Screen Inventory
8. Dashboard Philosophy
9. Search and Discovery Model
10. Enterprise Layout Standards
11. Design System Foundations
12. Page Templates
13. Accessibility and Responsive Standards
14. React Layout and Developer Standards
15. API and Permission Binding Principles
16. UX Definition of Done

1. Purpose and Scope

This handbook defines the UX foundation and information architecture for GuardianIQ. It does not replace detailed screen-by-screen specifications, but it establishes the rules that every future screen, workflow and component must follow. Developers should use it before building the Volume 2B screen specifications and before implementing the React frontend.

- Create a common UX language for GuardianIQ across AI registry, workflow, relationships, policy, risk, audit and administration modules.
- Define the navigation hierarchy and information architecture so teams do not create disconnected screens.
- Establish page layout patterns, component standards, states, permissions and accessibility requirements.
- Give frontend and backend developers a shared understanding of how UI behavior maps to entities, relationships, APIs and audit trails.
- Ensure GuardianIQ feels like an enterprise governance platform, not a collection of CRUD pages.

Out of scope: Pixel-perfect visual design, final Figma assets, theme token files and full React code are deferred to later implementation volumes. This document defines the architecture and behavior of the experience.

2. GuardianIQ UX Philosophy

GuardianIQ is an AI governance product. Its UX must do more than allow users to create records. The interface must help users understand what AI assets exist, what relationships they have, what governance controls apply, what actions are pending, what decisions were made and what evidence supports those decisions.

Principle	Meaning
Governance must be visible	Every critical page should show owner, status, risk level, relationship context and audit availability.
Explainability before action	Users should understand why an action is allowed, blocked, pending approval or escalated before they click.
Context must travel with the user	When a user moves from AI Agent to Workflow to Run History, the page must preserve entity context and breadcrumbs.
Enterprise density with progressive disclosure	Show enough information for professional users, but move deep details into drawers, tabs and drill-downs.
Audit by design	Every lifecycle screen should expose history, events or last updated context.
Human-in-the-loop clarity	Approvals, delegation, review responsibility and escalation paths must be explicit.
Relationship-first navigation	Users should be able to navigate through relationships, not only through menu items.
No hidden governance	If a policy, boundary, relationship or approval affects an action, surface it in the UI.

3. Product Design Principles

ID	Design Principle	Implementation Meaning
P1	Single source of truth	A user should never wonder whether the registry, workflow, relationship or audit view is authoritative. Each object should have one canonical details page.
P2	One object, many lenses	The same object can be viewed through registry, relationship, risk, audit and execution lenses. UI should provide tabs and cross-links rather than duplicating records.
P3	Action safety	Destructive, approval, execution and override actions require visible confirmation, reason capture and audit linkage.
P4	Role-aware experience	Menus, actions and editable fields must follow RBAC/ABAC decisions from backend, not just frontend role checks.
P5	Lifecycle clarity	Every governed object should show current state, allowed next states and blocked transitions.
P6	Relationship awareness	High-value screens must show connected entities: owner, workflow, policy, agent, tools, data sources, evidence and events.
P7	Readable complexity	Tables, graphs and timelines should use filters, grouping and summaries to make complex governance data usable.
P8	Recoverable mistakes	Validation errors should explain what is missing and how to fix it. Do not show raw

		backend errors to users.
P9	Performance visible	Long-running execution and search actions require progress states, background status and refresh patterns.
P10	Developer consistency	Every screen should use shared layout, table, badge, drawer, form and timeline components.

4. User Personas and Role-Based UX

GuardianIQ must support multiple enterprise personas. The same screen should not expose the same actions to every user. The frontend must use backend-provided permission decisions to determine visibility, editability and action availability.

Role-based UX model



Persona	Primary Need	Main Screens	UX Constraints
Executive	View portfolio health, risk, SLA, trends and governance readiness.	Command Center, Risk Dashboard, Audit Summary	Read-only, KPI driven, no detailed configuration by default.
Governance Admin	Configure registries, workflows, relationships, policies, approvals and controls.	Registry, Relationships, Policies, Administration	Broad configuration authority but high-risk activation may still require approval.
AI Asset Owner	Own models, agents and providers; respond to missing metadata and risk issues.	AI Registry, Relationship Explorer, Risk Findings	Can edit owned assets and see related runs and policies.
Workflow Designer	Create governed workflows, schedules and execution boundaries.	Workflow Registry, Designer, Schedule Wizard	Can create drafts; activation depends on risk and approval.
Risk / Compliance Officer	Review high-risk assets, policies, violations, approvals and audit evidence.	Risk, Policy, Approval Center, Audit Explorer	Can approve, reject, escalate based on scope.
Auditor	Review immutable timeline, evidence and decisions.	Audit Explorer, Evidence, Relationship Trace	Read-only and export oriented.

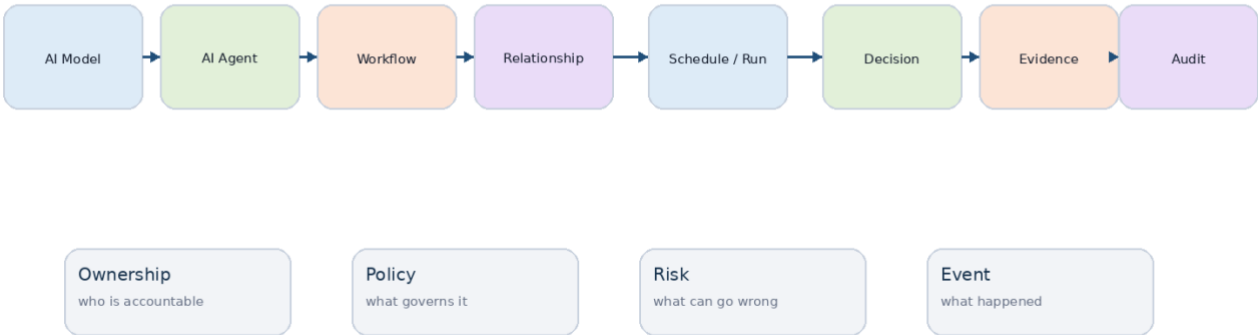
Operator	Monitor execution, failures, run outputs and notifications.	Execution Console, Run History, Notifications	Can retry or escalate only when authorized.
System Admin	Manage tenant configuration, users, roles and environment settings.	Administration, Settings	No automatic business approval powers unless assigned.

5. Information Architecture

GuardianIQ information architecture is organized around governable objects and the relationships among them. The UI must allow users to move from an object to its governance context without losing orientation.

Information architecture: governance object journey

Every screen should preserve context across AI assets, workflows, relationships, events and audit.



5.1 Core Information Objects

Object	Description	Must Show in UI
AI Provider	Source of model capability: vendor, internal, customer-owned or custom model provider.	Provider profile, risk, hosting, residency, contract, ownership.
AI Model	The model used to generate, classify, score or reason.	Model type, version, owner, provider, approval status, risk level.
AI Agent	Governed machine actor that can use models, tools and workflows.	Execution mode, boundaries, allowed tools, owner, violations.
Workflow	Business or governance process where AI participates.	Owner, department, criticality, related policies, schedules.
Relationship	Semantic link between objects such as owns, uses, governed by, supports or evidences.	Source, target, type, effective dates, status, validation state.
Schedule / Run	Execution configuration and each instance of governed execution.	Frequency, agent assignment, run status, outputs, failures.
Policy / Rule	Governance logic that controls recommendations, workflows and execution.	Version, conditions, actions, approvals, triggers.
Evidence / Audit	Artifacts and immutable events	Source, target, event type, actor, timestamp, payload.

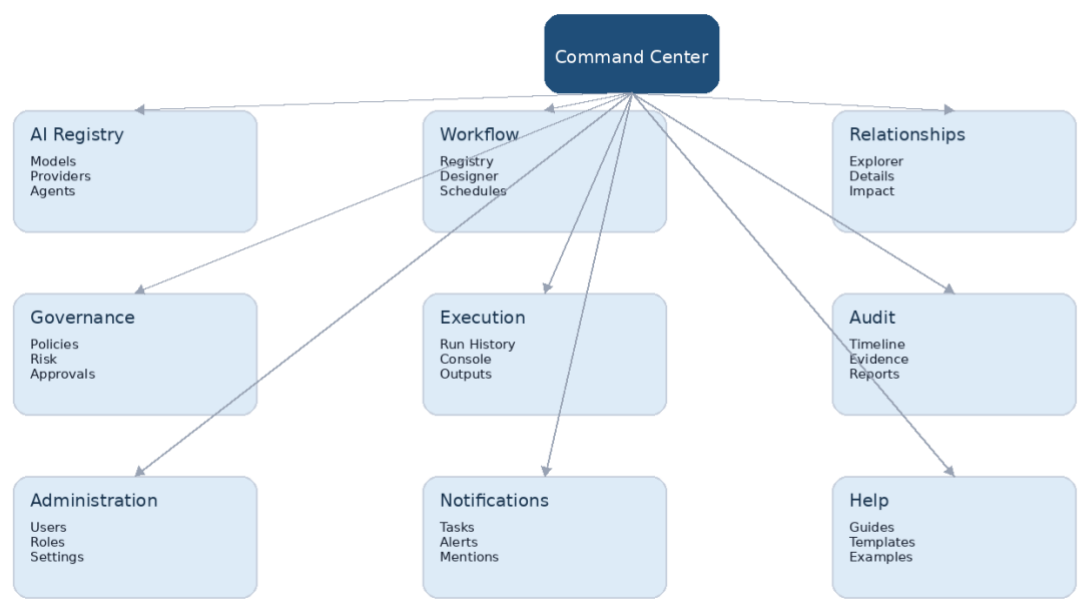
	supporting decisions.	
--	-----------------------	--

6. Navigation Architecture

The navigation model should be stable and shallow. Avoid hiding important modules under too many nested menus. Primary navigation should group screens by user intent.

GuardianIQ navigation architecture

Primary modules organized by user intent: govern, execute, review and administer.



Navigation Group	Routes	Purpose
Command Center	/dashboard	Executive and operational overview across governance, risk, execution and audit.
AI Registry	/ai-registry/models, /providers, /agents	Manage AI providers, models, agents and agent boundaries.
Workflow	/workflows, /workflow-designer, /workflow-scheduler	Create and operate governed workflows and schedules.
Relationships	/relationships/explorer, /relationships/:id	Explore, create, validate and audit relationships.
Governance	/policies, /risk, /approvals	Policy, risk and approval workspaces.
Execution	/execution/console, /runs	Monitor workflow runs, failures, outputs and retries.
Audit	/audit, /evidence	Review immutable events, evidence and traceability.
Administration	/admin/users, /roles, /settings	Manage platform, access and configuration.

6.1 Breadcrumb Standard

- Breadcrumbs must show entity context, not only route names.
- Format: Module > Object Type > Object Name / Code > Current Tab.
- Example: AI Registry > AI Agent > GOV_REVIEW_AGENT_001 > Relationships.
- Detail pages must include backlinks to related objects through canonical object URLs.
- If a user opens a page from global search, maintain the object breadcrumb and show a small “opened from search” hint only when useful.

7. Module Hierarchy and Screen Inventory

Module	Screens	Detailed Spec Volume
Command Center	Dashboard, Risk Summary, Execution Health, Approval Load, Audit Readiness	2B
AI Registry	Provider List, Provider Detail, Model List, Model Detail, Agent List, Agent Detail, Boundary Panel	2B
Workflow	Workflow List, Workflow Detail, Workflow Designer, Schedule Wizard, Schedule Detail	2B
Execution	Execution Console, Run History, Run Detail, Output Viewer, Failure Detail	2B
Relationships	Relationship Explorer, Relationship Create Wizard, Relationship Detail, Impact Analysis, Validation Results	2C
Governance	Policy List, Policy Detail, Risk Dashboard, Approval Center, Delegation Workspace	2C
Audit & Evidence	Audit Explorer, Event Detail, Evidence Viewer, Traceability Timeline	2C
Administration	User Management, Role Matrix, Permission Matrix, System Settings, Lookup Management	2C

8. Dashboard Philosophy

GuardianIQ should provide dashboards that help users decide what requires attention. Avoid dashboards that only show counts. Every dashboard widget should support drill-down to the underlying object list.

Dashboard	Primary Persona	Purpose	Recommended Widgets
Executive Dashboard	Executives	AI governance posture, open risks, high-risk agents, SLA breaches, audit readiness.	KPI cards, trend panels, risk heatmap, recent critical events.
Governance Dashboard	Governance Admins	Missing owners, pending approvals, invalid relationships, policy gaps.	Actionable work queue with filters.
Risk Dashboard	Risk / Compliance	High-risk models, violations, restricted data access, blocked actions.	Severity view, risk by department, escalation queue.
Execution Dashboard	Operators	Active schedules, running jobs, failed runs, retry queue, output health.	Live run status, failure trend, next run list.
Audit Dashboard	Auditors	Events by object, decision lineage, evidence completeness, export readiness.	Timeline summary, evidence gaps, audit exports.

9. Search and Discovery Model

Search and discovery model



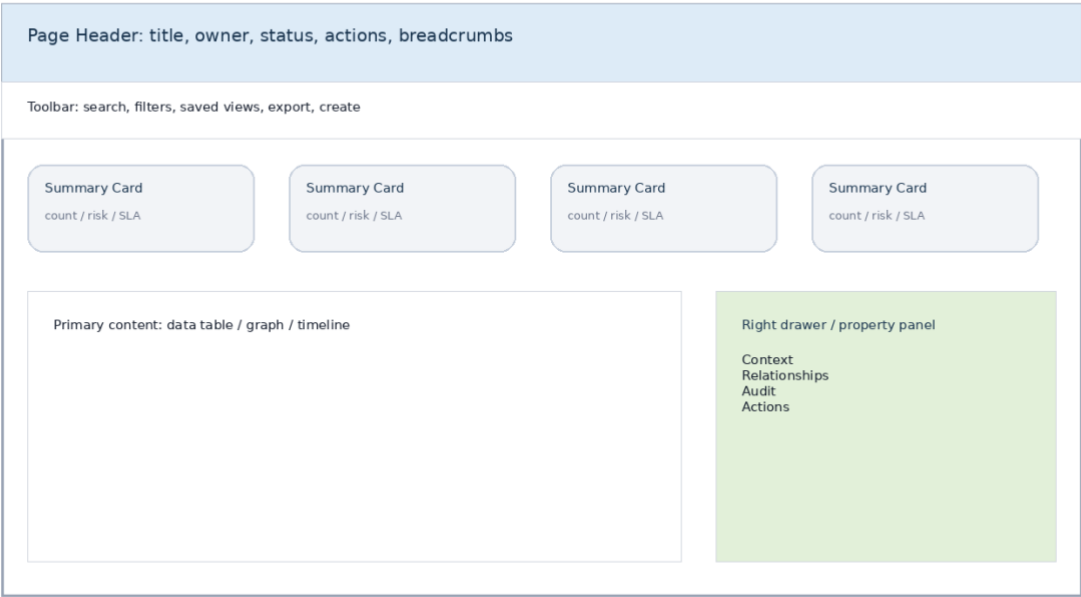
Developer rule: search results must expose object_type, object_id, display_name, status, risk_level, owner and last_event_at.

Search must support enterprise users who do not always know which module contains the object. Global search should cover object codes, names, owners, relationships, policies, events and evidence where authorized.

Search Type	Purpose	Required Result Metadata
Global Search	Search across modules.	Object type, code, name, owner, status, risk, last event.
Module Search	Search within a screen context.	Module-specific fields plus saved filters.
Relationship Search	Find connected objects.	Source, target, relationship type, status, effective dates.
Evidence Search	Find artifacts and audit support.	Evidence type, source system, target object, timestamp.
Workflow Search	Find workflows and schedules.	Workflow code, owner, department, next run, status.

10. Enterprise Layout Standards

Standard GuardianIQ page layout



10.1 Standard Page Anatomy

Area	Content	Usage
Page Header	Title, object code, lifecycle status, risk badge, primary actions, breadcrumbs.	All list and detail pages.
Toolbar	Search, filters, saved views, sort, export, create action.	List, explorer and history pages.
Summary Cards	Small set of context indicators.	Dashboards, details, execution pages.
Primary Content	Table, graph, timeline, wizard or console.	Every page.
Right Drawer / Property Panel	Contextual details, related objects, audit, validation and quick actions.	Relationship, registry, workflow and execution screens.
Footer / Meta Area	Last updated, source, pagination or system note.	Tables and long lists.

10.2 Responsive Rules

- Desktop-first enterprise layout should support 1366px, 1440px and 1920px widths.
- For tablets, collapse right-side drawers into full-width stacked panels.
- For mobile, provide read-only monitoring and approval-friendly layouts; complex graph editing can be desktop-only in early releases.
- Data tables must support horizontal scroll only as last resort; prioritize configurable columns and column hiding.
- Wizard steps should become vertical on narrow screens.

11. Design System Foundations

This section defines basic UI standards. A full component library will be covered in Volume 2D/2F, but developers should follow these foundations immediately.

Element	Standard	Developer Rule
---------	----------	----------------

Color	Use calm enterprise palette: navy for primary, blue for information, green for success, amber for warning, red for critical, gray for neutral.	Do not use decorative gradients on operational screens.
Typography	Use clear hierarchy: page title, section title, table header, body, helper text.	Avoid tiny text below 12px equivalent in dense tables.
Status Badges	Every lifecycle state should have a consistent badge.	DRAFT, ACTIVE, PENDING_APPROVAL, PAUSED, FAILED, RETIRED.
Risk Badges	Risk should be visible but not alarming unless critical.	LOW, MEDIUM, HIGH, CRITICAL.
Buttons	Primary button for one main action; secondary for alternatives; destructive actions require confirmation.	Never show more than 2 primary-style actions in one area.
Forms	Group fields by meaning, use inline validation, and show required fields clearly.	Validation comes from backend and frontend.
Tables	Support search, filters, sorting, pagination, column selection and row actions.	Default row density should be readable for enterprise users.
Drawers	Use for quick view, validation details, relationships and action panels.	Do not place long multi-step forms inside a narrow drawer.
Timelines	Use for audit, execution run steps and approval history.	Every item should show actor, event, time and summary.
Graph View	Use for relationships and impact analysis.	Always provide table alternative for accessibility.

12. Page Templates

GuardianIQ page templates



Template	Used For	Layout Pattern
List View	Registry lists, policy lists, workflows, runs, users.	Header + toolbar + table + row actions + detail drawer.
Details View	AI model, agent, workflow, policy, relationship detail.	Header + summary + tabs + relationship panel + audit timeline.
Wizard View	Create schedule, create relationship, register agent, submit policy.	Stepper + validation per step + review page + submit.
Explorer View	Relationship explorer, graph traversal,	Graph/tree + filters + property panel + table alternative.

	impact view.	
Console View	Execution monitoring and live run detail.	Status header + step timeline + logs + outputs + failure actions.
Audit View	Audit explorer, evidence viewer, event detail.	Search/filter + timeline + immutable event details + export.

13. Accessibility and Responsive Standards

Accessibility is not optional. GuardianIQ will be used for high-consequence governance activities, so users must be able to read, navigate, approve and audit without depending only on color or mouse interaction.

Area	Requirement	Implementation Note
Keyboard	All menus, forms, tables, wizards and modals must be keyboard navigable.	Tab order follows visual order; Esc closes modal/drawer.
Color contrast	Text, badges and interactive controls must maintain readable contrast.	Do not rely on color alone for risk/status.
Screen readers	Buttons, icons and graph nodes need accessible labels.	Use aria-label for icon-only controls.
Tables	Tables require headers, row labels and pagination controls accessible by keyboard.	Provide table alternative for graph data.
Forms	Error messages must be linked to fields.	Show clear text, not only red borders.
Focus states	Visible focus ring for all interactive elements.	Do not remove browser focus outline without replacement.
Responsive	Core read/review tasks must work on tablet.	Complex graph editing may be desktop only with notice.

14. React Layout and Developer Standards

The frontend should be implemented using a consistent module structure. Screen-specific logic must not be deeply mixed with reusable UI components.

14.1 Recommended Folder Structure

```
src/
  app/
    routes.tsx
    providers.tsx
  layouts/
    AppShell.tsx
    ModuleLayout.tsx
    DetailsLayout.tsx
  pages/
    dashboard/
    ai-registry/
    workflows/
    relationships/
    governance/
    execution/
    audit/
    admin/
  components/
    tables/
    forms/
    badges/
    drawers/
    timeline/
    graph/
    wizard/
```

```
filters/  
services/  
  apiClient.ts  
  aiRegistryService.ts  
  workflowService.ts  
  relationshipService.ts  
  auditService.ts  
hooks/  
  usePermission.ts  
  useEntityContext.ts  
  useTableState.ts  
  useAsyncAction.ts  
types/  
  entity.ts  
  relationship.ts  
  workflow.ts  
  audit.ts  
store/  
  sessionStore.ts  
  navigationStore.ts  
  filterStore.ts
```

14.2 Frontend Development Rules

- Pages own route-level orchestration; reusable components must remain domain-light where possible.
- All API calls must go through service modules, not directly from UI components.
- Permission logic must use backend decision responses or permission hooks; do not hardcode role names in JSX.
- Tables must use shared pagination, filtering and sorting state patterns.
- Forms must use shared validation rendering and consistent error display.
- All object links should use canonical object URL builder to avoid route drift.
- Every destructive action requires confirmation and reason capture where governance relevant.
- Every screen should have loading, empty, error and unauthorized states.

15. API and Permission Binding Principles

Principle	Meaning	Developer Guidance
Backend is authoritative	Frontend may hide actions for usability, but backend must enforce permissions.	Use permission endpoints and handle 403 responses gracefully.
Screen loads with context	Initial page load should fetch object, permissions, summary, relationships and latest events as separate or composed API responses.	Avoid over-fetching huge graphs on first load.
Standard response envelope	All APIs should return success, data, error, request_id and timestamp.	UI should show request_id in technical error details.
Permission-aware actions	Buttons should be visible/hidden/disabled based on action permissions and lifecycle state.	Show explanation for disabled governance actions.
Audit-aware changes	Create/update/approve/revoke actions should display success message with audit event reference where available.	Useful for troubleshooting and compliance.
Filters are query-state	List filters should be reflected in URL query params for shareability.	Works for saved views and PM/QA review.

16. UX Definition of Done

Area	Acceptance Criteria
Navigation	Screen is reachable from correct menu and breadcrumb is correct.

Permissions	Unauthorized actions hidden or disabled; backend 403 handled.
API Integration	Screen uses real service APIs or agreed mock contract with same response shape.
States	Loading, empty, error, success, unauthorized and validation states implemented.
Responsiveness	Layout works at 1366px and 1440px; tablet behavior defined if required.
Accessibility	Keyboard navigation, labels, focus states and color-independent statuses implemented.
Audit Context	Governance-relevant actions show event/history/timeline linkage.
Relationship Context	Screens for governed objects expose owner, status, risk and related objects.
QA Evidence	Test cases and screenshots captured for the implemented screen.
Developer Review	Component naming, folder structure, API mapping and error handling follow this handbook.

17. Immediate Implementation Checklist for Developers

1. Create AppShell with side navigation, top header, breadcrumbs and user/profile area.
2. Create shared PageHeader, Toolbar, DataTable, StatusBadge, RiskBadge, DetailsDrawer and Timeline components.
3. Create route groups for dashboard, ai-registry, workflows, relationships, governance, execution, audit and admin.
4. Implement permission guard hook that consumes backend permission/authorization response.
5. Implement canonical object link builder for AI_MODEL, AI_AGENT, WORKFLOW, RELATIONSHIP, POLICY, RUN and AUDIT_EVENT.
6. Implement consistent loading, empty and error components before building module screens.
7. Build the Dashboard shell, AI Registry shell, Workflow shell and Relationship Explorer shell using the page templates in this document.
8. Confirm with backend team that API response envelopes and object metadata fields align with this UX architecture.
9. Use this document as the baseline for Volume 2B detailed screen specifications.

18. Handover Notes

This Volume 2A establishes the UX and information architecture foundation. The next document, Volume 2B, should translate this architecture into screen-by-screen specifications for Dashboard, AI Registry, Workflow, Schedule, Execution and Notifications. Volume 2C should cover Relationship Explorer, Policy, Risk, Approval, Audit and Administration screens.

Implementation warning: Do not allow each module to invent its own layout or status language. GuardianIQ must feel like one enterprise platform, not multiple feature prototypes stitched together.